

Secure Code Review — Remediation Report

Project: CodeAlpha_SecureCodeReview

Task: CodeAlpha Cyber Security Internship — Task 3: Secure Coding Review

Author: Naima Rahmani

Target: app.py — intentionally vulnerable Flask application

Remediated target: app_fixed.py

Tools: Bandit v1.9.4 and Semgrep v1.171.0 (OSS, auto ruleset)

Before scan: 28 July 2026 **After scan:** 30 July 2026

1. Methodology

The vulnerable Flask application was reviewed using manual source inspection and automated SAST tools. Bandit v1.9.4 was executed against the original and remediated files. Semgrep v1.171.0 was executed using the auto ruleset. Findings were mapped to the intentionally introduced vulnerability classes and then remediated in app_fixed.py. The reported statistics below are taken from the actual terminal scan results supplied for this project.

2. Verified Scan Statistics

Metric	Before — app.py	After — app_fixed.py	Change
Bandit High	5	0	–5 (100% reduction)
Bandit Medium	5	0	–5 (100% reduction)
Bandit Low	5	3	–2 (40% reduction)
Bandit Total	15	3	–12 (80% reduction)
Semgrep findings	27	4	–23 (85.2% reduction)

Important interpretation: Bandit and Semgrep count rule findings differently. A single vulnerability can trigger multiple Semgrep rules, so the tool totals must not be interpreted as the number of unique vulnerabilities. The application intentionally contained 12 vulnerability classes.

3. Vulnerability Assessment and Remediation

ID	Vulnerability	Evidence / Rule	Remediation
VULN-01	Hardcoded Flask secret key	Bandit B105 — Low	Load FLASK_SECRET_KEY from environment; fail if absent.
VULN-02	Hardcoded DB credentials	Bandit B105 — Low	Move DB_USER/DB_PASSWORD to environment configuration.
VULN-03	SQL injection	Bandit B608 — Medium	Use a parameterized SQLite query with ? placeholders.
VULN-04	OS command injection (/ping)	Bandit B605 — High	Validate host input and execute argument list with shell=False.
VULN-05	Command injection (/backup)	Bandit B602 — High	Validate filename and execute tar without a shell.
VULN-06	eval() on user input	Bandit B307 — Medium	Remove eval(); use restricted AST-based arithmetic evaluation.
VULN-07	Unsafe pickle deserialization	Bandit B301 — Medium	Replace pickle.loads() with JSON parsing.
VULN-08	MD5 password hashing	Bandit B324 — High	Use Werkzeug's password hashing function instead of MD5.
VULN-09	Insecure random token	Bandit B311 — Low	Use Python secrets for cryptographically secure tokens.
VULN-10	SSRF / disabled TLS / no timeout	Bandit B501 High + B113 Medium; Semgrep after flags SSRF	Validate SSRF destination, reject private IPs, verify TLS, disable redirects, add timeout.
VULN-11	Reflected XSS	Semgrep before; Semgrep after flags HTML construction	Escape input as Jinja template data so autoescaping applies; length-limit input.
VULN-12	Path traversal	Semgrep before	Resolve requested path and require it to remain inside upload directory.
VULN-13	Debug mode / all-interface bind	Bandit B201 High + B104 Medium	Disable debug and bind to localhost by default.

4. Detailed Findings

SQL Injection

The original application concatenated the username directly into a SQL statement. The fixed version uses a parameterized query, separating SQL syntax from user-controlled data.

OS Command Injection

The original `/ping` endpoint used `os.system()` and `/backup` used `shell=True`. The fixed version uses `subprocess` argument lists with `shell=False` and validates the input.

Arbitrary Code Execution via eval()

The original `eval(expr)` could execute Python expressions supplied by a client. The fixed version parses an expression using AST and allows only a restricted set of numeric operations.

Insecure Deserialization

`pickle.loads()` was removed. JSON is used for the session payload, avoiding executable pickle objects.

Weak Password Hashing

MD5 was replaced with Werkzeug's password hashing implementation, which provides a password-oriented salted/adaptive hash.

Insecure Randomness

The predictable random module was replaced with `secrets.token_urlsafe()`, designed for security-sensitive tokens.

SSRF / TLS

The original endpoint accepted arbitrary URLs and disabled certificate verification. The fixed version validates the scheme and destination IP, verifies TLS, applies a timeout, and prevents automatic redirects.

Reflected XSS

The original endpoint inserted the name into the HTML source. The corrected implementation passes name as template data: `{{ name }}`. Jinja's autoescaping therefore treats HTML characters as text.

Path Traversal

The fixed endpoint resolves the requested path and checks that it remains within the configured upload directory before opening it.

Debug Exposure

Production-style execution now uses `debug=False` and localhost binding by default.

5. Post-Remediation Residual Findings

The final Bandit scan produced exactly 3 Low-severity, High-confidence findings: B404 for importing `subprocess` and B603 twice for `subprocess` calls without `shell=True`. These are defensive warnings rather than evidence that shell injection remains. The fixed code uses `shell=False` and validated argument lists. Therefore, the High and Medium Bandit findings were reduced from 10 to 0.

The final Semgrep scan produced 4 blocking findings: two SSRF-related rules on the URL-fetching code and two HTML/XSS-related rules on the `/greet` route. These findings are retained in the report rather than being claimed as fully eliminated. The code was manually reviewed: SSRF protections were added, but Semgrep's taint analysis still recognizes a request-controlled URL reaching `requests.get()`; the `/greet` route now uses Jinja

template variables and autoescaping, but the static rules still flag the dynamically rendered HTML pattern. This demonstrates why SAST results require manual validation.

6. Before/After Security Improvement

Bandit: 15 findings before → 3 after. This is an 80.0% reduction in total Bandit findings. More importantly, High severity fell from 5 to 0 and Medium severity fell from 5 to 0 — a 100% reduction in High and Medium findings. Only 3 Low defensive subprocess warnings remain.

Semgrep: 27 findings before → 4 after, a reduction of 23 findings or approximately 85.2%. The remaining 4 are documented rather than hidden or suppressed.

Do not combine the Bandit and Semgrep totals into one vulnerability count: the tools overlap and produce different rule-level findings. The meaningful conclusion is the reduction within each tool and the manual remediation of the 12 intentionally introduced vulnerability classes.

7. Reproduction Commands

```
bandit app.py
bandit app.py -f txt -o reports\bandit_before.txt
semgrep scan --config=auto app.py --json --output reports\semgrep_before.json
bandit app_fixed.py
bandit app_fixed.py -f txt -o reports\bandit_after.txt
semgrep scan --config=auto app_fixed.py --json --output reports\semgrep_after.json
```

8. Secure Coding Recommendations

1. Never commit secrets, passwords, API keys, or signing keys to source control.
2. Use parameterized queries or safe ORM APIs for database operations.
3. Avoid shell execution with user input; prefer direct APIs and shell=False when subprocesses are unavoidable.
4. Never use eval() or unsafe deserialization on untrusted input.
5. Use password-specific adaptive hashing rather than general-purpose hashes such as MD5.
6. Use cryptographically secure randomness for passwords, reset tokens, sessions, and other security-sensitive values.
7. Validate outbound URLs and network destinations to reduce SSRF risk, and always use TLS verification and timeouts.
8. Use framework autoescaping and safe template patterns to prevent XSS.
9. Canonicalize and constrain filesystem paths before opening user-selected files.
10. Disable debug mode in production and expose services only on required network interfaces.
11. Run SAST tools before committing or deploying code and manually review findings to distinguish true positives from defensive/static-analysis warnings.

9. Conclusion

The secure coding review demonstrated a complete identify → scan → remediate → rescan workflow. The vulnerable Flask application contained 12 intentionally introduced vulnerability classes. After remediation, Bandit confirmed that all High and Medium findings were eliminated, reducing its total findings from 15 to 3. Semgrep findings fell from 27 to 4. The remaining Semgrep findings were explicitly documented and manually assessed instead of being presented as a false zero-risk result. The project therefore demonstrates both secure coding remediation and responsible interpretation of SAST results.